RAG — Retrieval-Augmented Generation

A practical mini-curriculum for testing an ITI Trainer Copilot

1. What is RAG?

Retrieval-Augmented Generation (RAG) is a pattern where an AI system first retrieves relevant information from an external knowledge source, then gives that information to a language model as context for generating an answer.

Instead of asking the model to rely only on what it learned during training, RAG lets the application look up relevant documents at answer time. This is especially useful when the knowledge is private, domain-specific, frequently updated, or too large to place directly in a prompt.

2. The basic RAG pipeline

Step	What happens
1. User question	The user asks a question.
2. Query embedding	The question is converted into a vector representation.
3. Retrieval	A vector database finds chunks that are semantically similar.
4. Context building	The retrieved chunks are assembled into context.
5. Generation	The LLM answers using the retrieved context.
6. Sources	The application can expose document/page metadata for traceability.

Simple mental model: RAG = Search first → give evidence to the model → generate from that evidence.

3. Why is RAG useful?

• **Grounding:** answers can be tied to retrieved documents instead of relying only on model memory.

• **Private knowledge:** an application can retrieve information from internal manuals, courses, policies, or company documents.

• **Fresh knowledge:** documents can be updated without retraining the language model.

• **Traceability:** metadata such as file name, page, section, or slide can be returned with the answer.

• **Domain adaptation:** the same LLM can answer questions about a specialized knowledge base.

4. RAG vs. normal LLM prompting

Normal prompting	RAG
Question → LLM → Answer	Question → Retrieval → Context → LLM → Answer
Depends mainly on model knowledge + prompt	Uses external retrieved knowledge at runtime

Harder to cite private documents	Can attach source metadata
Knowledge changes require another strategy	Knowledge base can be updated independently

5. Core RAG concepts

Document loading: Read PDF, PPTX, DOCX, TXT, or Markdown files.

Chunking: Split documents into smaller pieces that can be retrieved independently.

Embeddings: Represent text as vectors so semantically related text can be compared.

Vector database: Store embeddings and retrieve similar chunks. Examples include ChromaDB, FAISS, and other vector stores.

Top-K retrieval: Return the K most relevant chunks for a query.

Context: The retrieved evidence passed to the generation model.

Grounded generation: Instruct the model to answer from the supplied evidence and avoid unsupported claims.

Metadata: Keep source, page, slide, section, course, and document type for traceability.

6. Example: ITI Trainer Copilot

Suppose the trainer asks: *“Create a 90-minute session about RAG for beginners.”*

1. Retrieve relevant curriculum chunks about RAG, embeddings, vector databases, and grounding.

2. Identify the learning objectives supported by the retrieved material.

3. Generate the session plan using only the retrieved curriculum as evidence.

4. Attach sources such as *RAG_Module.pdf, page 4*.

5. If the retrieved material does not support a claim, say that the curriculum does not provide enough information.

7. What can go wrong?

• **Bad retrieval:** the top chunks are related but do not directly answer the question.

• **Bad chunking:** important context is split across chunks.

• **Too few chunks:** the answer may be missing necessary evidence.

• **Too many chunks:** irrelevant context can distract the model.

• **Hallucination:** the model may add information not supported by the retrieved context unless the prompt and validation are designed carefully.

• **Duplicate information:** multiple retrieved chunks may come from the same section or page.

8. RAG quality checklist

■ Does retrieval return chunks relevant to the question?

■ Are source and page/slide metadata preserved?

■ Does the answer stay within the retrieved evidence?

■ Does the system clearly say when the evidence is insufficient?

■ Do broad and differently worded questions retrieve useful evidence?

■ Are retrieval failures measurable with a small evaluation set?

9. Mini hands-on lab

Goal: Build and test a minimal RAG pipeline over this document.

1. Load the PDF and extract its text.

2. Split it into chunks.

3. Create embeddings for each chunk.

4. Store the chunks and embeddings in ChromaDB.

5. For a user question, embed the query and retrieve the top 3–5 chunks.

6. Send the retrieved context to an LLM with a grounded-answer instruction.

7. Print the answer and the source metadata.

10. Suggested test questions

• What is RAG?

• Why is RAG useful?

• How does retrieval work in a RAG pipeline?

• What is an embedding?

• Why do we use a vector database?

• What is the difference between RAG and a normal LLM prompt?

• What happens when the retrieved context does not contain enough information?

Note for the ITI Trainer Copilot experiment: this document is intentionally focused on basic RAG concepts so it can serve as a clean test knowledge base. It is not intended to replace the project's actual ITI curriculum documents.